

EventTicketingPlatform

Architecture & Code Guide

Rebuilt from the updated repository: major user-facing parts, end-to-end behavior, architecture rationale, API, data, security, and deployment.

Repository	abdahmed22/EventTicketingPlatform
Snapshot commit	ab206fde34825b1c69b30ab3640c06c436ec9db3
Commit message	feature: add run steps for kubernetes
Commit date	12 August 2026, 09:16 (Africa/Cairo)
Guide date	12 August 2026

About this guide

This guide was regenerated from the latest default branch after the repository changed. It explains the system from the user interaction inward: Angular screens and services, HTTP and security, Spring controllers and transactional services, repositories and PostgreSQL, background scheduling, ticket PDF generation, and runtime infrastructure.

Scope

The document is based on static source inspection of commit `ab206fd`. Architectural reasons are inferred from the implemented behavior, comments, tests, framework conventions, and data constraints. Current mismatches are identified explicitly rather than described as intended design.

A “major part” is any screen, route, API, service, entity, job, security rule, persistence mechanism, or deployment element that a user interacts with directly or that materially changes the result of a user action by proxy.

Contents

1. System overview	3
1.1 Architecture and runtime	3
1.2 Roles	3
2. Repository and request path	5
3. Angular frontend	6
3.2 Routes and screens	6
3.5 Frontend services	7
4. Spring Boot backend	9
5. End-to-end workflows	11
6. Security model	14
7. Data model and state	15
8. HTTP API reference	17
9. Build, local runtime, Docker, and Kubernetes	19
10. Why the code is built this way	21
11. Tests and verification	22
12. Current implementation observations	23
13. File-finding guide and glossary	24

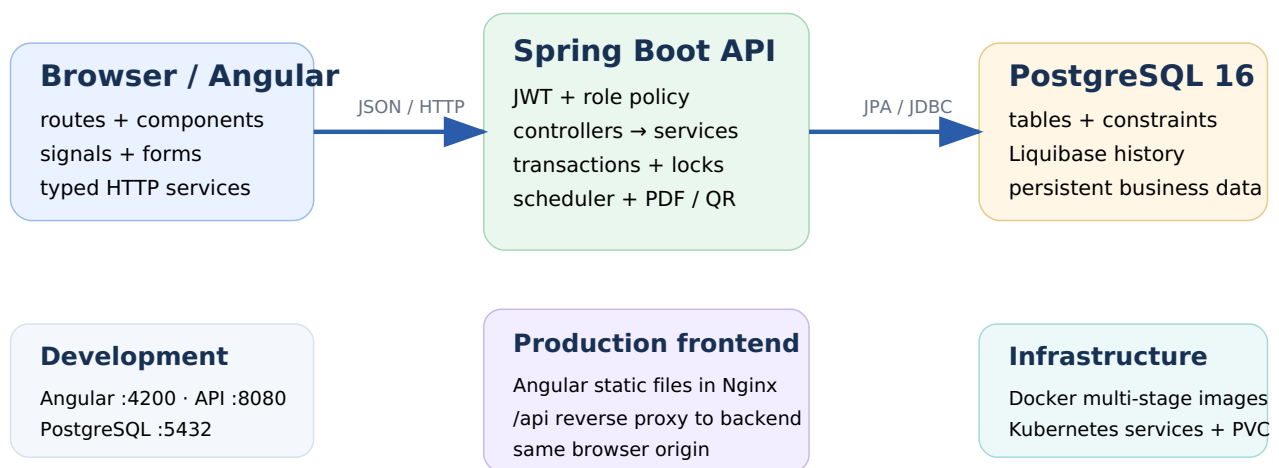
1. System overview

EventTicketingPlatform, branded “Eventix” in the UI, is a role-based event discovery, reservation, ticketing, and check-in system. Anonymous users discover published events. Customers reserve and confirm bookings. Organizers pass an approval process, request venues, publish events, manage inventory, and check in ticket holders. Administrators review applications and govern users, venues, events, and tickets.

Three processes are required locally

`ng serve` starts only Angular. Database-backed behavior requires PostgreSQL and the Spring Boot API. The browser never accesses PostgreSQL directly; it calls `http://localhost:8080/api`.

1.1 Architecture and runtime



The browser, API, and database are independent runtime responsibilities.

Area	Technology	Responsibility
Client	Angular 22.1, TypeScript 6, RxJS 7.8	SPA, standalone components, Signal Forms, lazy routes, typed API clients
API	Java 21, Spring Boot 4.1, Spring MVC	REST transport, business workflows, validation, scheduling
Security	Spring Security, JWT, BCrypt	Stateless bearer authentication and role policy
Persistence	Spring Data JPA, Hibernate, PostgreSQL 16	Transactions, queries, row locks, durable relational data
Schema	Liquibase	Ordered database creation and evolution
Tickets	OpenPDF and ZXing	A5 ticket PDF and QR code generation
Delivery	Docker, Nginx, Kubernetes	Reproducible builds, static serving, proxy, probes, storage

1.2 Roles

Actor	Primary capabilities
Anonymous visitor	Browse/filter published events, view public details, create a customer account, or submit an organizer application.
Customer	Reserve seats, watch the hold countdown, confirm or cancel, inspect bookings/tickets, and download the ticket PDF.

Actor	Primary capabilities
Organizer	Manage own venue requests and events, create seat tiers, publish/cancel, inspect bookings, list tickets, and check in a party.
Administrator	Review organizer/venue applications, manage users, view/cancel events, create approved venues, inspect/download/void tickets.
Background scheduler	Expires overdue PENDING holds and returns seats. Users experience its effect indirectly.

2. Repository and request path

2.1 Top-level layout

```
EventTicketingPlatform/
├─ client/           Angular single-page application
├─ Server/           Spring Boot API and Liquibase migrations
├─ Infrastructure/    PostgreSQL Compose and Kubernetes manifests
└─ README.md         Architecture summary and local/Kubernetes run steps
```

The monorepo keeps all application layers together, but it does not make them one executable. Angular, Spring Boot, and PostgreSQL have independent dependency managers, build outputs, ports, health, and deployment lifecycle.

2.2 Normal request lifecycle

#	Layer	What happens
1	Angular component	A route load, form, button, timer, or filter initiates work.
2	Angular feature service	Builds the URL, query, body, and expected response type.
3	HTTP interceptors	Add bearer credentials and normalize failures into <code>ApiError</code> .
4	Spring Security	Applies CORS, JWT validation, authentication, and coarse role policy.
5	Controller	Parses path/query/body data, validates DTOs, and delegates.
6	Service	Applies ownership, state, price, inventory, and transaction rules.
7	Repository	Executes CRUD, filtering JPQL, or pessimistic locked reads.
8	PostgreSQL	Persists records and enforces keys, uniqueness, and relationships.
9	Response	DTO JSON returns; Angular updates signals and the visible screen.

2.3 Why separate these responsibilities

Components stay focused on visible state and user interaction. Angular services keep endpoint knowledge out of templates. Controllers define the HTTP contract without becoming business objects. Services provide testable policy and atomic transactions. Repositories contain persistence syntax and lock semantics. DTOs protect secrets and prevent accidental entity graph serialization. Security and error handling are centralized because every feature needs them.

3. Angular frontend

3.1 Bootstrap, shell, and environments

File	Purpose
client/src/main.ts	Bootstraps the standalone App using appConfig.
app/app.ts and app.html	Navigation header, route outlet, and footer.
app/app.config.ts	Router, XHR HttpClient, auth interceptor, error interceptor, and global error listeners.
environment.ts	Development API URL: http://localhost:8080/api .
environment.prod.ts	Production API URL: /api , handled by Nginx.

The relative production path prevents browser code from knowing a cluster-internal hostname. Nginx owns service discovery and proxying, while the browser remains same-origin.

3.2 Routes and screens

Route	Guard	Component and behavior
/	Public	EventListComponent — paginated catalog and filters.
/events/:id	Public/role-aware	EventDetailComponent — public booking UI or organizer/admin controls.
/login	Guest	LoginComponent.
/register	Guest	RegisterComponent for CUSTOMER accounts.
/register/organizer	Guest	OrganizerApplicationComponent.
/bookings	Redirect	Redirects to /customer/bookings .
/customer/bookings	Authenticated	The feature-rich booking component is now routed: filters, countdown, confirm, cancel, and PDF.
/customer/bookings/:id	Authenticated	BookingDetailComponent with confirm/cancel and direct single-ticket loading.
/tickets	Redirect	Redirects to /customer/tickets .
/customer/tickets	Customer	MyTicketsComponent using principal-derived ticket APIs.
/customer/tickets/:ticketCode	Customer	TicketDetailComponent and PDF download.
/organizer/events	Organizer/Admin	Own events, or all events for an admin.
/organizer/events/new	Organizer/Admin	Create DRAFT event.
/organizer/events/:id/edit	Organizer/Admin	Edit manageable event.
/organizer/venues	Organizer/Admin	Venue submit/list/edit; admin creates approved venues.
/organizer/tickets	Organizer	Event ticket list and check-in; now linked in desktop/mobile navigation.
/admin/events	Admin	All event states and cancellation.
/admin/tickets	Admin	Ticket oversight, PDF, lookup, and void.
/admin/organizer-applications	Admin	Application approval/rejection.

Route	Guard	Component and behavior
/admin/venues	Admin	Venue approval/rejection.
/admin/users	Admin	User role and deletion management.
**	Fallback	Unknown routes return to the catalog.

Important change in this snapshot

The previous routing mismatch was corrected. `/customer/bookings` now loads `components/booking/my-bookings`, the component that exposes countdown, confirmation, cancellation, filters, and PDF download. Event reservation now navigates directly to the created booking's detail route.

3.3 Signals, forms, and asynchronous work

Signals hold screen-local data such as events, bookings, errors, loading states, active filters, selected categories, modal visibility, and action-in-progress IDs. Computed values derive filtered bookings, selected ticket tiers, ticket views, and display names. Signal Forms provide validation for login, registration, organizer applications, events, venues, and seat categories.

HttpClient returns RxJS Observables. Components subscribe and update signals in success and error callbacks. Form submissions sometimes use `firstValueFrom`. `forkJoin` combines tickets, bookings, events, and users when a compact ticket payload needs readable display data. The booking screen subscribes to a one-second interval and unsubscribes during destruction.

3.4 Authentication state, guards, and interceptors

AuthService stores the JWT and user summary in localStorage and exposes token, user, role, and computed role flags. The navigation renders links by role. The auth interceptor adds `Authorization: Bearer ...`. The error interceptor maps the backend envelope into `ApiError` and clears the session on a 401 received by a logged-in user.

Route guards improve user experience, but they are not authoritative security. Browser code can be bypassed, so Spring Security and service-level ownership checks remain mandatory.

3.5 Frontend services

Service	Boundary
AuthService	Registration, login, persisted session, role signals.
EventService	Public, organizer, and administrator event reads and commands.
VenueService	Approved list, organizer submissions, admin create/review/CRUD.
SeatCategoryService	Public tier list and organizer create/update.
BookingService	Reserve, confirm, cancel, own bookings, and event bookings.
TicketService	Principal-based ticket list/single/PDF, legacy admin PDF path, organizer check-in, admin actions.
OrganizerApplicationService	Guest submission and administrator review.
UserAdminService	User list, detail, role change, and deletion.
DashboardService	Pending queue API; no routed dashboard component currently exists.

3.6 User-facing component behavior

Catalog and event detail

The catalog sends optional category, venue, date, price, page, and size filters. Event detail selects public, organizer, or admin data based on role and shows ticket inventory, management controls, or a booking widget.

Booking management

The routed booking list displays one-second countdowns, updates expired state, filters by status, confirms/cancels, and downloads confirmed tickets through a principal-based endpoint.

Ticket screens

Customer ticket list/detail now obtains identity from the JWT instead of placing the customer UUID in normal customer URLs. Ticket data is enriched with booking and public event data.

Organizer and admin

Organizers manage venues/events and can reach Event Tickets from navigation and event rows. Administrators review queues, manage users, inspect events, and oversee tickets.

4. Spring Boot backend

4.1 Bootstrap and configuration

`EventTicketing.Application` uses `@SpringBootApplication` and `@EnableScheduling`. Component scanning discovers controllers, services, repositories, the JWT filter, startup runner, and scheduler. Java 21 is required by the Maven compiler configuration.

Configuration	Effect
<code>spring.datasource.*</code>	PostgreSQL address and credentials.
<code>spring.liquibase.change-log</code>	Master schema migration file.
<code>ddl-auto: none</code>	Hibernate does not invent or alter tables; Liquibase owns schema evolution.
<code>security.jwt.*</code>	HMAC signing secret and 24-hour expiry.
<code>app.cors.allowed-origins</code>	Permitted frontend origins.
<code>admin.seed.*</code>	Initial administrator account.
<code>management.endpoints.*</code>	Health, readiness, and liveness endpoints.

4.2 Layer responsibilities

Layer	Major files	Responsibility
Controllers	Auth, organizer application, venue, event, seat category, booking, ticket, user, dashboard	HTTP mapping, DTO validation, principal injection, status codes.
Services	Auth, organizer application, venue, event, seat category, booking, ticket, PDF, user	Business rules, ownership, state transitions, transaction boundaries.
Repositories	JpaRepository interfaces and custom JPQL	CRUD, filters, existence checks, updates, and row locks.
Models	JPA entities and status enums	Persistent state and relationships.
DTOs	Request/response records	Explicit API contract and validation without exposing secrets or lazy graphs.
Cross-cutting	SecurityConfig, JwtFilter, GlobalExceptionHandler	Authentication, authorization, CORS, and consistent errors.

4.3 Controllers

Controllers parse path variables, optional filters, and request bodies, then delegate to services. Authentication principals are injected as User entities. Resource creation returns 201, deletion returns 204, and normal reads/updates return 200. TicketController was extended in the new snapshot with `/api/tickets/my` routes and explicit 404 behavior when admin, organizer, or customer ticket lookups return null.

4.4 Services and transactions

Service	Protected behavior
AuthService	Uniqueness, BCrypt password storage, authentication, JWT issue.
OrganizerApplicationService	Pending state and transactional ORGANIZER account creation.
VenueService	Ownership, review state, admin creation, linked-event delete guard.
EventService	Future dates, approved venues, owner access, publish/cancel rules, filtering.

Service	Protected behavior
SeatCategoryService	Tier creation and total/available seat consistency.
BookingService	Inventory locks, hold, price, confirm/cancel/expire, event cascade.
TicketService	Idempotent issue, check-in, void, owner/admin PDF authorization.
TicketPdfService	Trusted A5 PDF and QR artifact.
UserService	Admin-only role changes/deletion and final-admin protection.

4.5 Validation and errors

Request records use Jakarta validation for required fields, email, phone patterns, string length, price, quantity, and capacity. GlobalExceptionHandler maps known failures into a stable envelope with timestamp, status, error code, message, and field errors.

HTTP	Code	Typical meaning
400	VALIDATION_ERROR / INVALID_PARAMETER / INVALID_STATE	Malformed data or illegal transition.
401	INVALID_CREDENTIALS / INVALID_TOKEN	Login or JWT failure.
403	FORBIDDEN_ACTION / ACCESS_DENIED	Role, ownership, or business permission failure.
404	RESOURCE_NOT_FOUND	Missing entity, hidden public state, or absent ticket.
409	DUPLICATE_RESOURCE / SEAT_UNAVAILABLE	Unique conflict or inventory race.
500	INTERNAL_ERROR	Unexpected error is logged without exposing internals.

5. End-to-end workflows

5.1 Registration and login

The registration form validates identity and password confirmation, then AuthService posts to `/api/register`. The backend normalizes blank phone to null, checks unique email and phone, BCrypt-hashes the password, and saves a CUSTOMER user.

Login accepts email or phone. Spring AuthenticationManager verifies credentials, JwtService signs a token whose subject is the user UUID, and the frontend persists token plus user summary. Every protected request receives the token through the interceptor.

Why: BCrypt keeps plaintext passwords out of storage. Stateless JWTs avoid server-side session storage. Reloading the user by token subject on each request makes current database roles authoritative.

5.2 Organizer onboarding

A guest submits an application. The proposed password is hashed immediately, and the application remains separate from users while PENDING. Admin approval creates an ORGANIZER with the existing hash in the same transaction, then records reviewer and review time. Rejection records its reason. A reviewed application cannot be reviewed again.

Why: unapproved applicants never receive organizer authority, but approval does not require a second registration or temporary plaintext password.

5.3 Venue approval

An organizer submits a venue as PENDING. An administrator approves or rejects it and records the reviewer. Public venue lists return only APPROVED venues. The event form offers approved venues, and EventService repeats the approval check during creation and publication.

Why: a restricted dropdown is helpful but bypassable. The server check is the invariant. A venue cannot be hard-deleted while an event references it.

5.4 Event and seat-category lifecycle

Action	Rule and effect
Create	Future date/time and approved venue required; status is always DRAFT.
Add seat tier	Owner/admin only; available seats begin equal to total seats.
Update seat total	Reserved count remains constant; total cannot fall below already reserved seats.
Publish	DRAFT only; venue still approved; future date; at least one tier with seats.
Browse	Only PUBLISHED events are exposed publicly.
Cancel	Moves to CANCELLED and resolves active bookings/tickets.
Hard delete	Admin only and blocked when bookings exist.

5.5 Public event discovery

EventListComponent translates filters into query parameters. EventRepository runs pageable JPQL and left-joins seat categories for price filtering. DISTINCT removes duplicate event rows when multiple tiers match, and an explicit count query supplies pagination totals. Public detail requires the requested event itself to be PUBLISHED.

5.6 Reservation and concurrency

No external payment provider

Confirmation acts as the checkout/payment boundary. Reservation creates a temporary PENDING hold; confirmation changes it to CONFIRMED and issues admission.

#	BookingService reserve behavior
1	Load event and require PUBLISHED state.
2	Load the seat category under a PESSIMISTIC_WRITE lock.
3	Require category/event match, available quantity, and per-person limit.
4	Calculate <code>price × quantity</code> on the server.
5	Decrease <code>availableSeats</code> inside the same transaction.
6	Create a PENDING booking expiring five minutes later.

When a competing buyer takes the final seat, the server returns a 409. The event screen refreshes availability. The updated snapshot sends a successful reservation directly to `/customer/bookings/{bookingId}`, where confirmation and cancellation are visible.

Why: browser checks prevent obvious mistakes, but only a database transaction and row lock can serialize simultaneous buyers and stop overselling.

5.7 Countdown, confirmation, cancellation, and expiry

The routed My Bookings component updates a one-second countdown for every PENDING hold. When a timer reaches zero, it marks the local card expired for immediate feedback and reloads from the authoritative backend.

Confirmation and cancellation lock the booking row before inspecting state. Confirmation refuses an expired or non-PENDING hold, saves CONFIRMED, and generates the ticket. Cancellation returns seats, saves CANCELLED, and voids an already issued ticket. Booking detail now exposes the same actions.

The scheduler runs every minute, finds overdue PENDING bookings, passes IDs to a transactional method, re-fetches each under a lock, rechecks state, locks the seat tier, returns seats once, and saves EXPIRED. The own-bookings read path also expires overdue records, reducing visible scheduler delay.

5.8 Ticket issuance and principal-derived access

TicketService issues one ticket per confirmed booking. It checks for an existing ticket before creation, while a unique database constraint on `booking_id` provides the final guarantee. The ticket snapshots event, venue, tier, owner, quantity, total, code, and status.

Safer customer API introduced

Normal customer screens now call `GET /api/tickets/my`, `GET /api/tickets/my/booking/{bookingId}`, and `GET /api/tickets/my/booking/{bookingId}/pdf`. Identity comes from the verified JWT principal instead of a customer UUID supplied by the browser. Legacy customer-ID paths remain for compatibility and administrator downloads.

TicketController also converts missing ticket results to explicit 404 responses instead of allowing null mapping to become an internal error.

5.9 PDF and check-in

TicketPdfService creates an A5 document containing holder, event, date/time, venue, seat category, quantity, booking ID, price, status, issue time, ticket code, and QR code. PDF authorization requires the booking owner or an administrator.

An organizer selects an event, views issued tickets, and checks in a code. The service requires code and event to match, rejects CANCELLED and CHECKED_IN tickets, then records CHECKED_IN and the time. Because one ticket covers the booking quantity, check-in admits the represented party.

5.10 Event cancellation cascade

Original state	Cascade effect
PENDING booking	Seats returned and booking becomes EXPIRED.
CONFIRMED booking	Seats returned, booking becomes CANCELLED, and ISSUED ticket is voided.
Already CANCELLED/EXPIRED	Skipped to prevent double inventory release.

Each booking and seat category is locked before mutation. This costs more queries than a bulk update but preserves the same invariants as individual booking actions.

6. Security model

6.1 Authentication sequence

1. Angular attaches the JWT as a bearer token.
2. JwtFilter extracts the user UUID from the signed token subject.
3. CustomUserDetailsService reloads the user by UUID.
4. JwtService verifies signature, subject match, and expiration.
5. Spring Security stores the User principal and current ROLE authority.
6. Path matchers enforce role groups.
7. Services enforce resource ownership and state-specific permissions.

6.2 Coarse path policy

Access	Paths
Public	Registration, login, organizer application, health, and GET <code>/api/events/**</code> .
Customer or admin	<code>/api/customer/**</code> and <code>/api/tickets/customer/**</code> .
Organizer or admin	<code>/api/organizer/**</code> and <code>/api/tickets/organizer/**</code> .
Admin	<code>/api/admin/**</code> and <code>/api/tickets/admin/**</code> .
Customer, organizer, or admin	New <code>/api/tickets/my</code> and <code>/api/tickets/my/**</code> principal-derived routes.
Any authenticated user	Everything else, notably <code>/api/bookings/**</code> .

The API is stateless, CSRF is disabled, CORS permits configured origins and headers, and authentication/access-denied handlers produce the frontend's standard JSON error shape.

6.3 Why principal-derived ticket routes matter

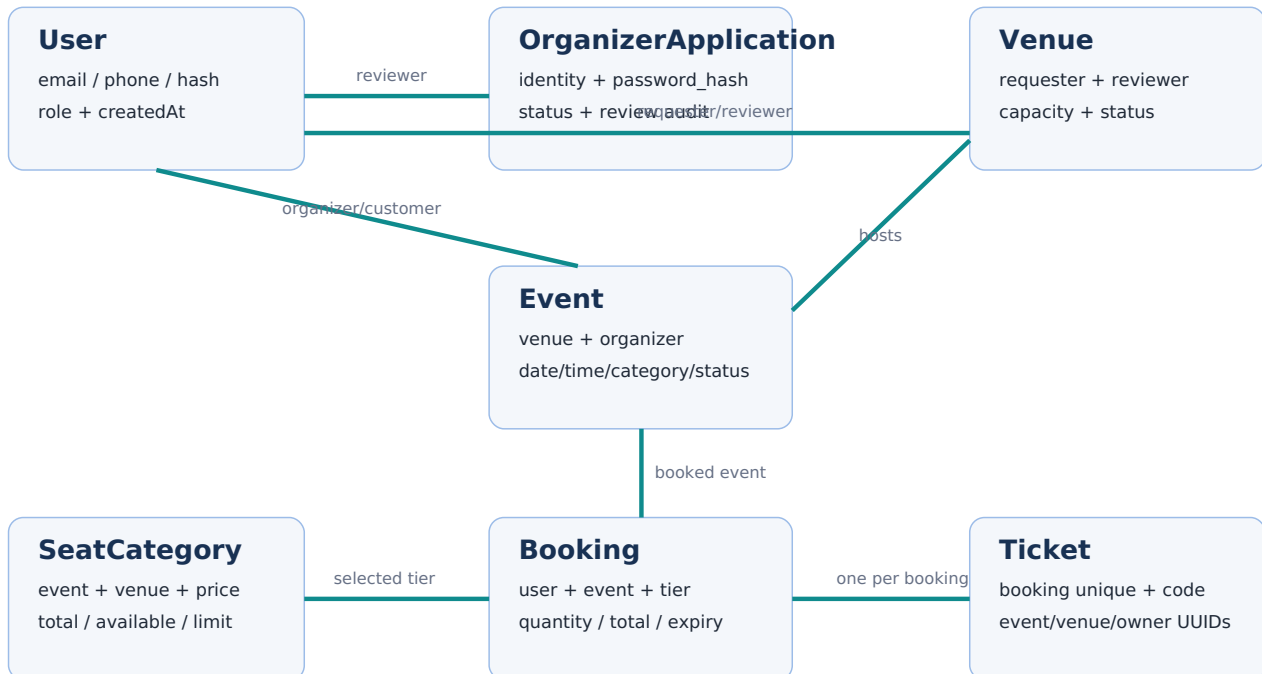
A browser-provided customer UUID is untrusted input even when the controller checks it. The new `/my` endpoints remove that parameter from normal customer operations entirely. The server selects the account from the validated principal, reducing accidental identity mix-ups and making the contract simpler.

6.4 Production cautions

LocalStorage is simple and persists across reloads, but a successful XSS attack could read its token. Real deployment requires strong content security controls and HTTPS. Committed database, JWT, and administrator values are development defaults and should be rotated and managed outside source control.

7. Data model and state

7.1 Main entities



Simplified relationships. TicketAttendee exists as an additional table but is not active in current UI workflows.

Table	Meaning	State or key invariant
users	Accounts and Spring principal data	CUSTOMER / ORGANIZER / ADMIN
organizer_applications	Pre-account approval record	PENDING / APPROVED / REJECTED
venues	Locations requested and reviewed	PENDING / APPROVED / REJECTED
events	Organizer-owned event at a venue	DRAFT / PUBLISHED / CANCELLED
seat_categories	Price tier and inventory	totalSeats, availableSeats, seatingCapacity
booking	Temporary hold or purchase	PENDING / CONFIRMED / CANCELLED / EXPIRED
tickets	One admission artifact per booking	ISSUED / CHECKED_IN / CANCELLED
ticket_attendee	Potential ticket-to-customer link	Present but not used by current routes

7.2 Persistence choices

UUIDs are opaque API identifiers and avoid a central numeric sequence. Enums are stored as strings for readability and ordinal safety. Lazy entity relations avoid loading full graphs unless DTO mapping needs them. Event keeps both raw venue/organizer UUID fields and read-only entity relations to access identifiers without always initializing lazy objects.

Ticket stores related UUID snapshots rather than full JPA relations. This keeps the ticket record compact and independent but makes customer/admin screens perform enrichment requests. Database uniqueness reinforces email, phone, ticket code, and one-ticket-per-booking assumptions.

7.3 Liquibase

Change	Purpose
001	users
002	organizer applications
003	venues
004	events
005	seat categories
006	booking
007	booking indexes
007 ticket change set	tickets and indexes
008	ticket attendee
009	ticket quantity, deduplication, and unique booking reference

Liquibase is the schema source of truth because Hibernate DDL generation is disabled. Migration order follows foreign-key dependencies, and DATABASECHANGELOG prevents repeat application.

8. HTTP API reference

8.1 Public and authentication

Verb	Path	Purpose
POST	/api/register	Create CUSTOMER.
POST	/api/login	Authenticate and return JWT plus user summary.
POST	/api/register/organizer-application	Submit organizer application.
GET	/api/events	Filter/page PUBLISHED events.
GET	/api/events/{id}	Published event detail.
GET	/api/events/venues	Approved venues.
GET	/api/events/{eventId}/seat-categories	Seat tiers.
GET	/actuator/health/**	Health and probes.

8.2 Booking and principal ticket APIs

Verb	Path	Access and purpose
POST	/api/bookings	Authenticated; create five-minute PENDING hold.
POST	/api/bookings/{id}/confirm	Owner; confirm and issue ticket.
POST	/api/bookings/{id}/cancel	Owner; cancel and return inventory.
GET	/api/bookings/my	Principal's bookings; expire overdue holds.
GET	/api/tickets/my	Principal's tickets without a user ID parameter.
GET	/api/tickets/my/booking/{bookingId}	Single issued ticket for owned booking.
GET	/api/tickets/my/booking/{bookingId}/pdf	Owner/admin PDF download.
GET	/api/tickets/customer/{customerId}	Legacy self/admin ticket list.
GET	/api/tickets/customer/{customerId}/{ticketCode}	Legacy self/admin ticket lookup.
GET	/api/tickets/customer/{customerId}/booking/{bookingId}/pdf	Legacy owner/admin PDF path.

8.3 Organizer

Verb	Path	Purpose
GET/POST	/api/organizer/events	List owned events or create DRAFT.
GET/PUT	/api/organizer/events/{id}	Manage owned event.
POST	/api/organizer/events/{id}/publish	Publish valid draft.
POST	/api/organizer/events/{id}/cancel	Cancel and cascade.
GET	/api/organizer/events/{eventId}/bookings	Event bookings.
POST	/api/organizer/events/{eventId}/seat-categories	Create tier.
PUT	/api/organizer/seat-categories/{id}	Update tier.
GET/POST	/api/organizer/venues	List own venues or submit request.

Verb	Path	Purpose
PUT	/api/organizer/venues/{id}	Update owned venue.
GET	/api/tickets/organizer/{eventId}	List event tickets.
GET	/api/tickets/organizer/{eventId}/{ticketCode}	Ticket for event; missing result now returns 404.
POST	/api/tickets/organizer/events/{eventId}/check-in	Check in ticket code.

8.4 Administrator

Area	Endpoints	Purpose
Users	GET /api/admin/users , GET/PATCH/DELETE /api/admin/users/{id}	List/detail, role change, delete.
Applications	GET /api/admin/organizer-applications , approve/reject POSTs	Review organizer applications.
Venues	GET/POST /api/admin/venues , update/approve/reject/delete by ID	Full venue governance.
Events	GET /api/admin/events , detail/update/delete/cancel by ID	Every event state.
Bookings	GET /api/admin/bookings	All bookings.
Tickets	UUID lookup, customer/event lists, edit, cancel	Ticket oversight and voiding.
Dashboard	/api/admin/dashboard and nested review actions	Pending queue aggregate; no Angular route yet.

9. Build, local runtime, Docker, and Kubernetes

9.1 Requirements

Tool	Version/use
Java JDK	21; Maven wrapper builds and runs backend.
Node.js	22.x recommended by container build; npm installs frontend.
Docker	Runs PostgreSQL locally and builds Kubernetes images.
PostgreSQL	16 via Docker or compatible local service.
kubectI / Minikube	Local Kubernetes deployment.

9.2 Local development

```
# PostgreSQL from repository root
sudo /usr/bin/docker compose -f Infrastructure/docker-compose.yaml up -d

# Backend in another terminal
cd Server
chmod +x mvnw
./mvnw spring-boot:run

# Frontend in another terminal
cd client
npm install
npm start
```

PostgreSQL listens on 5432, Spring Boot on 8080, and Angular on 4200. If the host maps PostgreSQL to 5433, override `SPRING_DATASOURCE_URL=jdbc:postgresql://localhost:5433/eventticketing`. Startup runs Liquibase, seeds the initial administrator when absent, and starts the booking expiry scheduler.

Compose scope

`Infrastructure/docker-compose.yaml` starts only PostgreSQL. The API and Angular must still be started separately.

9.3 Docker images

The backend Dockerfile uses a Java 21 JDK build stage and a smaller Java 21 JRE runtime under a non-root user. The frontend Dockerfile uses Node 22 to build Angular and Nginx to serve static output. Multi-stage builds keep compilers and build caches out of runtime images.

9.4 Kubernetes flow documented by the updated repository

```
minikube start
eval $(minikube docker-env)
docker build -t eventticketing-server:local ./Server
docker build -t eventticketing-client:local ./client
kubectl apply -f Infrastructure/k8s/namespace.yaml
kubectl apply -f Infrastructure/k8s/database/
kubectl apply -f Infrastructure/k8s/backend/
kubectl apply -f Infrastructure/k8s/frontend/
kubectl get pods -n event-ticketing
kubectl get svc -n event-ticketing
```

The frontend is NodePort 30000 and the backend is 30080. PostgreSQL remains internal and uses a 1 GiB ReadWriteOnce PVC. Backend probes use Actuator health; PostgreSQL probes use `pg_isready`. The Nginx frontend proxies `/api/` to the backend's cluster DNS name.

The configured CORS origin includes the default Minikube frontend address `http://192.168.49.2:30000`. If `minikube ip` differs, the backend allowed origin must be changed and its image rebuilt.

9.5 Initial data

A new database contains no sample venues or events. Public browsing is empty until an organizer and venue are approved and an event is configured and published. The development administrator defaults to `admin@example.com` and `Admin12345678Admin`; these are not production-safe credentials.

10. Why the code is built this way

Decision	Reason	Trade-off
Angular SPA + REST API	Separates interaction design from domain/data policy and allows independent deployment.	Development needs multiple processes and a stable contract.
Standalone lazy components	Less module boilerplate and smaller initial bundle.	Imports are distributed among components.
Signals + feature services	Lightweight reactive UI state and centralized HTTP boundaries.	No single global state history.
Layered Spring architecture	Transport, policy, and persistence are testable independently.	More files and mapping than direct CRUD.
DTO contracts	Validation, secret protection, controlled JSON, no lazy graph leakage.	Mapper code is required.
Stateless JWT	Independent API replicas and simple bearer authentication.	No refresh/revocation flow; localStorage requires XSS protection.
Principal-derived / tickets/my	Removes spoofable customer identity from normal customer URLs.	Legacy paths remain for compatibility/admin behavior.
Role paths plus ownership checks	Readable coarse policy plus resource-specific protection.	Ownership must be applied consistently in every service.
PostgreSQL + Liquibase	Strong relational integrity and controlled schema history.	Integration behavior needs PostgreSQL.
Pessimistic row locks	Reliable prevention of overselling and double release.	High contention serializes requests for one tier.
Five-minute hold	Checkout time without permanent inventory loss.	Scheduler and race-safe expiry are required.
One ticket per booking	Simple group artifact, PDF, uniqueness, and idempotent issue.	Individual partial check-in is not modeled.
Server PDF + QR	Consistent trusted artifact independent of browser layout.	Adds dependencies and server CPU work.
Nginx same-origin proxy	Stable browser /api URL and internal Kubernetes discovery.	Proxy configuration is environment-specific.

The central backend concern is consistency rather than basic CRUD. Organizer/venue approval gates publication; publication gates booking; booking atomically changes inventory; exactly one operation resolves a hold; confirmation issues exactly one ticket; cancellation returns inventory and voids admission. Transactions, locks, state enums, and database constraints implement those invariants.

The frontend is optimistic about presentation but not authority. It validates early, disables invalid buttons, calculates display totals, and shows countdowns. The backend repeats all important rules because browser data can be stale or bypassed.

11. Tests and verification

Test	Style	Coverage
EventServiceTest	Mockito unit	Draft create, venue approval, publish conditions, cancel cascade.
VenueServiceTest	Mockito unit	Pending submit, list, approve, linked-event delete guard.
SeatCategoryServiceTest	Mockito unit	Create inventory, total delta, reserved-seat floor.
EventRepositoryTest	Spring/PostgreSQL	Published date-range filtering.
BookingConcurrencyTest	Spring/PostgreSQL threads	Last seat, no oversell, confirm/expire and cancel/expire races.
EventControllerTest	Reflection smoke	Expected controller method names.
Angular specs	Component smoke	Mostly component creation in this snapshot.

The concurrency tests are the architecturally important suite because mocks cannot prove database locks. Source comments state they had not been executed in the audit environment in which they were written, so they require a real run before being treated as verified.

```
cd Server
./mvnw test

cd ../client
npm test
```

Useful additions include Testcontainers, controller/security tests, end-to-end role workflows, route/button assertions for the corrected booking flow, and migration tests from both an empty and an older database.

12. Current implementation observations

Observation	Impact	Category
Booking UI integration was fixed	The rich booking list is now routed, reservation opens booking detail, and confirmation/cancellation are visible.	Resolved prior gap
Principal ticket routes were added	Customer list, single-ticket, and PDF operations no longer require the frontend to send its own user UUID.	Security/contract improvement
Organizer ticket navigation was added	Desktop/mobile navigation and event rows now expose Event Tickets.	Resolved discoverability gap
Ticket not-found responses improved	Selected admin/organizer/customer lookups now produce 404 instead of null mapping failures.	Error handling improvement
Admin dashboard route remains missing	Backend API and DashboardService exist, and mobile navigation links there, but the route falls through to home.	Frontend integration gap
Organizer ticket ownership remains service-unchecked	Role policy exists, but list/get/check-in does not verify that the authenticated organizer owns the supplied event ID.	Security gap
Venue rejection reason is not persisted	The DTO accepts a reason, but the Venue entity/service has no stored field.	Data/UX gap
Dashboard review APIs duplicate dedicated APIs	Two controller surfaces perform organizer and venue review actions.	API duplication
Ticket display needs enrichment	Denormalized ticket UUIDs lead to extra booking/event/user requests.	Performance trade-off
TicketAttendee is dormant	Schema/entity/service pieces exist, but no active route manages individual attendees.	Partial feature
Cancelled event enrichment can disappear	Ticket screens use the public event endpoint, which hides cancelled events; display detail may be absent.	Presentation gap
Admin full ticket edit is API-only	The UI supports lookup/download/void, not arbitrary row editing.	Unused API surface
Some admin filtering is in memory	User/venue findAll filtering should become repository pagination as data grows.	Scalability concern
User deletion may meet foreign-key errors	Role checks exist, but related event/venue/booking references are not preflighted.	Error handling gap
Compose remains database-only	The README now explains local and Kubernetes startup, but local full-stack orchestration is still manual.	Operations trade-off
Development secrets are committed	Database, JWT, and administrator values must be rotated and externalized.	Production risk

12.1 Recommended priorities

1. Enforce organizer ownership in ticket list, specific-ticket, and check-in methods.
2. Externalize and rotate all committed deployment credentials.
3. Add or remove the missing admin dashboard route and align its navigation.
4. Persist venue rejection reason and merge duplicate review APIs.
5. Paginate admin lists and consider enriched ticket DTOs to reduce client joins.
6. Clarify whether TicketAttendee and direct full-row ticket editing remain requirements.

13. File-finding guide and glossary

13.1 Where to change behavior

Task	Primary location
Routes and guards	<code>client/src/app/app.routes.ts</code> and <code>guards/*</code>
Role navigation	<code>client/src/app/components/layout/nav-header/*</code>
API base URL	<code>client/src/environments/environment*.ts</code>
Bearer/error behavior	<code>client/src/app/interceptors/*</code>
Feature HTTP calls	<code>client/src/app/services/<feature>/*</code>
Endpoint roles and CORS	<code>Server/.../configuration/SecurityConfig.java</code>
REST contract	Matching controller, DTO, Angular model, and Angular service
Business rules	<code>Server/.../service/*Service.java</code>
Queries and locks	<code>Server/.../repository/*Repository.java</code>
Schema	<code>Server/src/main/resources/db/changelog/changes/*</code>
Local PostgreSQL	<code>Infrastructure/docker-compose.yaml</code>
Production proxy	<code>client/nginx.conf</code>
Kubernetes	<code>Infrastructure/k8s/**/*</code>

13.2 Glossary

Term	Meaning here
DTO	Explicit request or response shape separate from persistence entities.
JWT	Signed bearer token whose subject is the user UUID.
JPA/Hibernate	Java object-relational mapping to PostgreSQL.
Liquibase	Versioned schema migration runner.
Pessimistic write lock	Database row lock that makes competing writers wait.
Signal	Angular reactive value that updates dependent views.
Observable	RxJS representation of HTTP or timer work.
State machine	Allowed status transitions and their guards.
Idempotent	A retry does not produce a second effect, such as a duplicate ticket.
Reverse proxy	Nginx receives browser <code>/api</code> calls and forwards them to the backend service.

Final mental model

Angular presents and coordinates. Spring Security identifies. Controllers translate. Services decide and transact. Repositories query and lock. PostgreSQL persists and constrains. Liquibase evolves. The scheduler repairs expired holds. Ticket and PDF services turn a confirmed booking into admission.